

企业级应用软件设计与开发大作业报告

项目名称: **AutoSpider**: 面向网页数据采集的纯视觉浏览器智能
体

课程名称: 企业级应用软件设计与开发

方向: Agentic AI 原生开发

学号: 2025303024

姓名: 刘博林

专业: 软件工程

指导教师: 戚欣

提交日期: 2026 年 6 月 22 日

目录

- 一、选题背景与设计思想..... 4
 - 1.1 问题定义 4
 - 1.2 现有方案不足..... 4
 - 1.3 项目价值 4
 - 1.4 技术路线 4
- 二、Specs 规格文档 5
 - 2.1 Product Spec 5
 - 2.2 用户故事 6
 - 2.3 非功能需求..... 6
 - 2.4 Architecture Spec..... 6
 - 2.5 API / CLI Spec..... 7
 - 2.6 配置规格 7
- 三、系统架构与设计..... 8
 - 3.1 系统总体架构..... 8
 - 3.2 Agent 交互流程..... 10
 - 3.3 数据流设计..... 11
 - 3.4 状态管理与记忆机制..... 12
 - 3.5 关键设计取舍..... 12
- 四、关键实现与代码展示..... 13
 - 4.1 任务澄清与结构化..... 13
 - 4.2 Memory Checkpoint 改造..... 13
 - 4.3 Memory URL Channel 13
 - 4.4 Playwright 浏览器工具..... 13
 - 4.5 OpenAI 兼容 API 适配..... 14
 - 4.6 Windows 启动脚本 14
 - 4.7 配置安全实践..... 14
- 五、测试与评估..... 16
 - 5.1 测试环境 16

5.2 功能测试	17
5.3 Agent 行为评估	17
5.4 课程核心技术要素覆盖	18
5.5 当前问题分析	18
5.6 Demo 结果说明	18
六、系统升级与扩展	19
6.1 可扩展架构	19
6.2 下一阶段计划	19
6.3 AI 能力演进路径	19
七、课程总结	20
7.1 个人收获	20
7.2 工程思维转变	20
7.3 对课程的理解	20
附录：仓库目录结构	20
参考资料	21

一、选题背景与设计思想

1.1 问题定义

网页数据采集是企业级应用软件中常见的数据获取方式。企业在做竞品分析、内容聚合、舆情监控、商品数据同步或行业研究时，经常需要从公开网页中提取结构化信息。传统爬虫通常依赖开发者手写 XPath、CSS Selector 或接口解析逻辑，这类方案在页面结构稳定、字段固定时效率较高，但在复杂页面、动态渲染页面和临时性采集任务中维护成本明显上升。

本项目希望解决的问题是：如何让用户只通过自然语言描述采集目标，就能让 AI Agent 自动理解网页、规划采集步骤、操作浏览器、收集详情页链接并抽取结构化字段。项目以 AutoSpider 为基础，构建一个面向网页数据采集任务的纯视觉浏览器智能体。

1.2 现有方案不足

- 固定规则爬虫依赖人工编写选择器，页面结构变化后需要重新维护。
- 通用 LLM 对话只能给出建议，无法直接操作浏览器和访问真实页面。
- 简单网页解析工具缺少任务规划能力，难以处理多步骤页面跳转和详情页抽取。
- 传统自动化脚本通常面向单一网站，难以复用到新的采集场景。
- 缺少完整可观测链路，失败时不容易定位是页面导航、URL 收集还是字段抽取环节出现问题。

1.3 项目价值

本项目的核心价值在于将网页采集从“规则编写”转化为“目标描述”。用户不需要预先分析网页 DOM，也不需要手动编写 XPath。系统通过 LLM 和浏览器自动化工具将采集目标拆解为可执行步骤。

- 对用户：降低采集任务配置门槛，适合快速验证小规模数据采集需求。
- 对开发者：提供一个可扩展的 Agentic AI 工程样例，包含任务澄清、规划、工具调用、状态管理和结果落盘。
- 对课程项目：覆盖 Agentic AI 原生开发的核心技术要素，具备完整运行链路和可展示 Demo。

1.4 技术路线

- Agent 框架：LangGraph + LangChain，用于多阶段状态流和 LLM 调用。
- 浏览器工具：Playwright，用于打开网页、截图、点击元素、访问详情页。
- 视觉理解：Set-of-Mark 标注可交互元素，结合多模态 LLM 判断页面操作。
- 状态管理：LangGraph checkpoint，支持中断、恢复和阶段状态追踪。
- 本地运行：memory checkpoint + memory URL channel，降低 Redis 部署依赖。
- 数据落盘：SQLite + JSON / JSONL 文件，保存任务计划、URL、抽取结果和摘要。
- 演示场景：Bangumi 每日放送页动画条目采集。

从工程实现角度看，本项目没有把 LLM 仅作为“文本生成器”，而是将其放在可执行系统的决策节点中。LLM 负责理解页面截图、选择下一步动作、判断采集字段和生成结构化结果；Playwright、数据库、文件系统和状态机负责执行与持久化。这样的设计更符合 Agentic AI 原生开发的思路：模型不是独立回答问题，而是在受控工具环境中完成一个有状态任务。

本项目的课程定位是方向一“Agentic AI 原生开发”。它不是对某个已有业务系统做界面包装，而是围绕网页数据采集这个明确任务，从零组织 Agent 执行链路，包括需求澄清、规格设计、架构拆分、运行配置、关键代码改造、测试评估和后续扩展计划。

二、Specs 规格文档

2.1 Product Spec

模块	说明	优先级
自然语言任务输入	用户输入采集目标、目标网页、字段和数量	P0
任务澄清	LLM 解析 URL、字段、目标数量、最大翻页数	P0
任务规划	根据页面和字段生成采集计划与子任务	P0
浏览器导航	Playwright 打开页面、截图、识别并点击元素	P0
URL 收集	从列表页收集详情页 URL	P0
字段抽取	访问详情页并抽取中文标题、日文标题、星期等字段	P0
结果落盘	输出 merged_results.jsonl、summary、runtime.log	P0
历史任务复用	对相同网站和相似任务复用已有经验	P1
Redis 扩展	支持 Redis channel 和 checkpoint，用于更大规模采集	P2

2.2 用户故事

- 作为用户，我希望输入“从 Bangumi 每日放送页采集 5 条动画，字段包括中文标题、日文标题、放送星期”，系统能够自动打开网页并输出结构化结果。
- 作为用户，我希望系统在开始执行前展示识别到的任务配置，避免错误采集。
- 作为开发者，我希望每次运行都有日志、计划文件和成功率摘要，方便定位失败原因。
- 作为课程展示者，我希望系统不依赖 Redis 也能在本地完成一个小规模端到端 Demo。

2.3 非功能需求

类别	需求说明	当前实现
可运行性	普通 Windows 环境可启动，不强制依赖服务器部署	提供 .cmd 启动脚本和 memory 模式
可配置性	API Key、模型地址、数据库地址通过环境变量配置	使用 .env 管理，避免硬编码密钥
可观测性	执行阶段、错误、模型请求、采集结果可追踪	输出 runtime.log、task_plan.json、summary
可恢复性	Agent 中断后可以保留上下文并继续执行	支持 LangGraph checkpoint
可扩展性	后续可以切换 Redis、增加 worker 或接入前端	保留 Redis pipeline 与上下文分层
安全性	不把 API Key 写入 GitHub 仓库	.gitignore 排除 .env 和 output

2.4 Architecture Spec

系统采用分层结构组织：

层级	目录	职责
Interface	interface/cli	CLI 命令、用户交互、运行入口
Composition	composition	LangGraph 工作流、pipeline 调度、结果聚合
Chat Context	contexts/chat	任务澄清、字段定义、需求结构化

Planning Context	contexts/planning	页面分析、任务规划、子任务生成
Collection Context	contexts/collection	浏览器导航、URL 收集、详情页抽取
Experience Context	contexts/experience	Skill、XPath、历史任务经验沉淀
Platform	platform	LLM、Playwright、SQLite、日志、配置

2.5 API / CLI Spec

入口	类型	说明
start_bangumi_demo.cmd	Windows 脚本	固定 Bangumi 小规模演示任务
start_autospider.cmd	Windows 脚本	交互式自然语言采集入口
autospider chat-pipeline	CLI	核心采集命令，支持 request、target-url-count、max-pages 等参数
autospider resume	CLI	从 LangGraph checkpoint 恢复线程
autospider doctor	CLI	检查运行环境和依赖
autospider benchmark	CLI	Benchmark 测试入口

CLI 入口的设计目标是让 Demo 和调试都保持可复现。对于课程展示，优先使用 `start\_bangumi\_demo.cmd`，因为它固定了目标页面、目标数量和本地 memory 模式，能够减少现场输入错误；对于后续扩展，则可以使用 `autospider chat-pipeline` 直接输入新的网页采集需求。

2.6 配置规格

配置项	作用
BAILIAN_API_KEY	OpenAI 兼容 API Key

BAILIAN_API_BASE	OpenAI 兼容 API Base URL
BAILIAN_MODEL	使用的模型名称
DATABASE_URL	SQLite 数据库地址
GRAPH_CHECKPOINT_ENABLED	是否启用 LangGraph checkpoint
GRAPH_CHECKPOINT_BACKEND	checkpoint 后端，当前 Demo 使用 memory
PIPELINE_MODE	URL channel 模式，当前 Demo 使用 memory
PLAYWRIGHT_BROWSERS_PATH	Playwright 浏览器下载目录

配置文件不提交到 GitHub。报告和 README 只展示变量名与示例格式，不展示真实 Key。这一点对课程项目也很重要：Agent 项目通常需要外部模型服务，如果把 Key 写入代码或 README，仓库公开后会造成直接泄露风险。

三、系统架构与设计

3.1 系统总体架构

系统整体由 CLI 入口、LangGraph 编排、领域上下文和平台工具组成。CLI 接收用户输入，Chat Context 将自然语言需求转为结构化任务，Planning Context 生成执行计划，Collection Context 调用 Playwright 浏览器完成页面操作，最终将数据写入 output 目录。

图 3-1 AutoSpider 系统总体架构

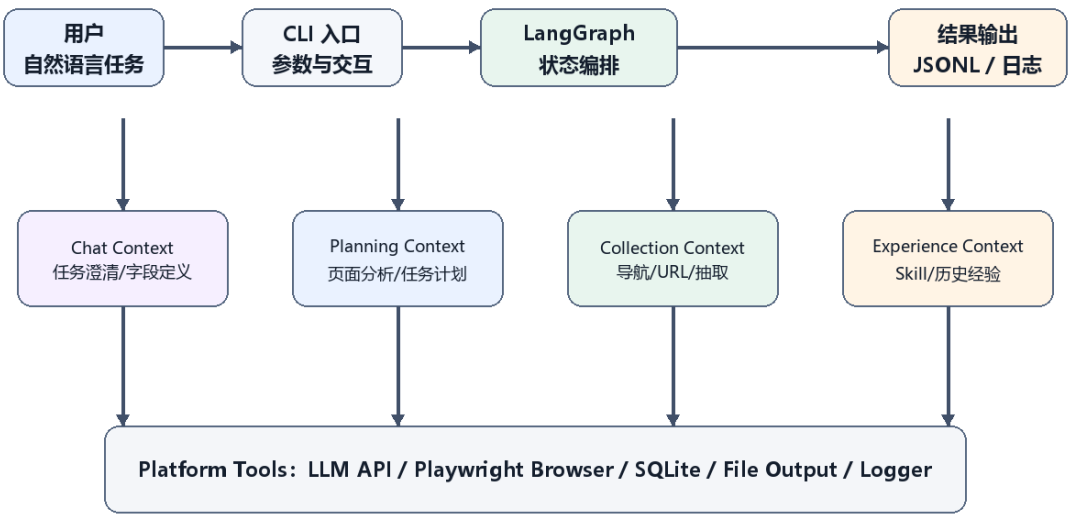


图 3-1 AutoSpider 系统总体架构

User Request

|

v

Interface CLI

|

v

LangGraph Workflow

|

+--> Chat Context -> LLM Task Clarification

+--> Planning Context -> Page Understanding / TaskPlan

+--> Collection Context -> Playwright / URL / Field Extraction

+--> Experience Context -> Skill / XPath / History Reuse

|

v

Output Artifacts: JSONL / JSON / Log / SQLite

3.2 Agent 交互流程

图 3-2 Agent 执行流程

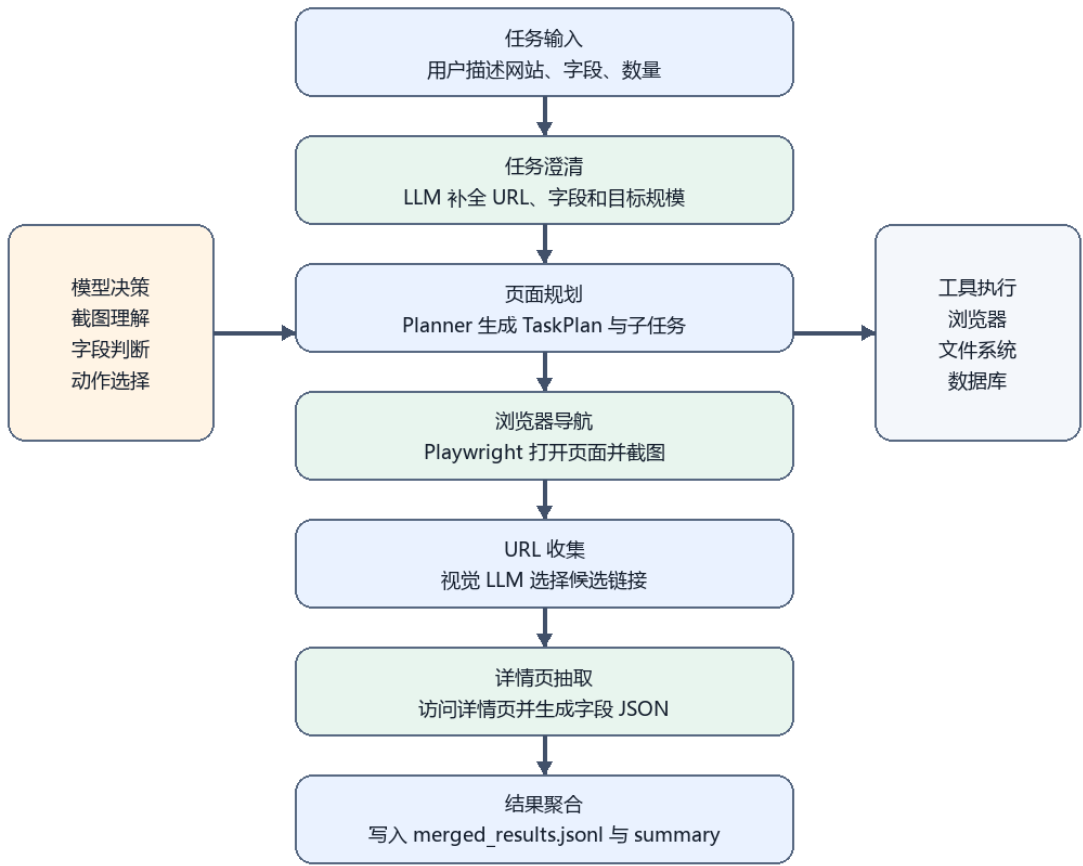


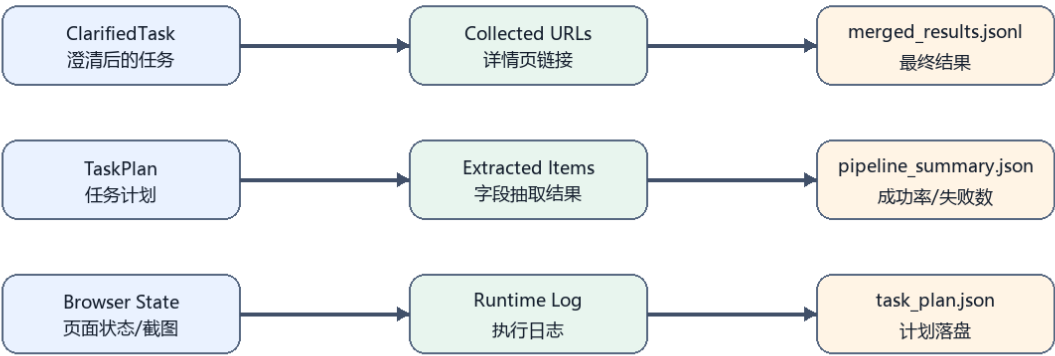
图 3-2 Agent 执行流程





3.3 数据流设计

图 3-3 数据流与产物关系



状态对象进入 LangGraph；执行产物写入 output 目录，便于复盘、调试和评估。

图 3-3 数据流与产物关系

数据对象	说明	输出位置
ClarifiedTask	澄清后的任务配置	LangGraph state
TaskPlan	任务计划和子任务结构	output/task_plan.json

Collected URLs	收集到的详情页链接	output/subtask_leaf_001/collected_urls.json
Extracted Items	字段抽取结果	output/subtask_leaf_001/pipeline_extracted_items.jsonl
Merged Results	合并结果	output/merged_results.jsonl
Pipeline Summary	成功率、失败数、状态	output/subtask_leaf_001/pipeline_summary.json
Runtime Log	全链路日志	output/runtime.log

3.4 状态管理与记忆机制

系统使用 LangGraph 管理多阶段状态，支持 interrupted、success、failed 等状态。原项目主要使用 Redis 作为 checkpoint 和队列后端；本项目为课程演示增加了 memory 模式，使 Demo 在本机无 Redis 的情况下也能运行。

memory 模式适合小规模课程展示。Redis 模式仍保留在代码中，可用于后续多 worker、多任务并发和更稳定的持久化执行。

状态管理设计解决了两个问题。第一，Agent 执行过程可以被拆分为多个可观察阶段，不再是一个不可解释的长调用。第二，当模型提出澄清问题或任务中断时，系统可以保留已有上下文，而不是重新开始整条链路。课程演示阶段采用 memory 后端，是在“完整工程能力”和“本地可运行性”之间做出的折中。

3.5 关键设计取舍

设计点	可选方案	本项目选择	原因
页面理解	DOM 解析 / 视觉截图	纯视觉为主	更接近人类浏览页面方式，适合动态页面
浏览器控制	Selenium / Playwright	Playwright	异步能力好，截图和定位能力更完整
状态后端	Redis / Memory	Demo 使用 Memory	降低部署复杂度，保证本地演示可运行
结果展示	Web 前端 / 文件输出	文件输出	当前重点是 Agent 执行链路，不虚构前端
任务规模	大规模并发 / 小规模稳定 Demo	小规模稳定 Demo	课程展示更关注端到端闭环和可解释性

四、关键实现与代码展示

4.1 任务澄清与结构化

任务澄清阶段的目标是将用户输入转为明确的采集配置。系统会识别目标 URL、字段名、字段描述、目标数量和最大翻页数，并在 CLI 中展示给用户确认。

示例任务：

```
Collect 5 anime entries from Bangumi calendar page https://bangumi.tv/calendar.  
Fields: Chinese title, Japanese title, weekday.  
Target URL count 5.
```

4.2 Memory Checkpoint 改造

为了降低本地部署复杂度，项目增加 memory checkpoint 后端。核心思想是根据环境变量选择 checkpoint 实现：

```
GRAPH_CHECKPOINT_ENABLED=true  
GRAPH_CHECKPOINT_BACKEND=memory
```

当使用 memory 后端时，LangGraph 状态保存在进程内存中，适合课堂 Demo。对于生产环境，可切换为 Redis 后端。

4.3 Memory URL Channel

原项目 pipeline 主要依赖 Redis channel 分发 URL 任务。本项目新增 memory URL channel，在本地串行执行时直接使用内存队列传递 URLTask。

```
PIPELINE_MODE=memory  
LOCAL_SERIAL_MODE=true  
PIPELINE_CONSUMER_CONCURRENCY=1
```

该设计减少了 Redis 服务配置成本，同时保留了 Redis 扩展能力。

4.4 Playwright 浏览器工具

Playwright 是本项目的核心工具调用能力，负责：

- 打开 Bangumi 日历页面。
- 等待页面加载。
- 获取页面截图。
- 标注可交互元素。

- 根据 LLM 决策点击候选链接。
- 进入详情页抽取字段。

4.5 OpenAI 兼容 API 适配

在实际部署中，不同 API 供应商对 OpenAI SDK 默认请求头的兼容性不同。本项目增加兼容请求头处理逻辑，减少第三方 API 供应商拦截请求的概率，提升本地运行成功率。

4.6 Windows 启动脚本

脚本	作用
start_bangumi_demo.cmd	固定 Bangumi 演示任务，适合课堂展示
start_autospider.cmd	交互式输入采集目标

脚本统一设置 UTF-8、Playwright 浏览器路径、SQLite 数据库路径、memory checkpoint 和 memory pipeline，减少手动命令错误。

```
cd src\autospider-project
start_bangumi_demo.cmd
```

该脚本适合作业展示，因为它减少了 PowerShell 执行策略、虚拟环境激活、环境变量拼写等问题。开发者仍然可以直接使用 CLI 入口进行更自由的实验。

4.7 配置安全实践

项目运行依赖外部模型 API。为了避免密钥泄露，`.env` 文件只保留在本地，不提交到 GitHub。仓库中只提供配置项说明和示例，真实 Key 通过环境变量读取。这样即使仓库公开，也不会暴露模型服务凭证。

```
BAILIAN_API_KEY=请在本地填写
BAILIAN_API_BASE=https://your-provider.example.com/v1
BAILIAN_MODEL=your-model-name
```

这种方式符合企业级应用中的基本安全要求：配置与代码分离，敏感信息不进入版本库。

4.8 关键代码展示

本节展示本项目中与 Agent 执行、状态管理、本地 Demo 启动直接相关的关键代码。代码片段来自 src/autospider-project，能够对应前文的 memory checkpoint、memory URL channel 和 Windows 演示脚本。

代码 4-1 CLI 图执行入口 (src/autospider/interface/cli/_runtime.py)

```
def invoke_graph(entry_mode: str, cli_args: dict[str, Any], *, thread_id: str = ""):
```

```

if entry_mode != "chat_pipeline":
    raise ValueError(f"unsupported entry mode: {entry_mode}")
_ensure_database_ready()
graph_result = run_async_safely(
    _build_run_chat_pipeline().run(cli_args=dict(cli_args), thread_id=thread_id)
)
result = graph_result.model_dump()
_log_graph_runtime(result)
return result

```

该入口将 CLI 参数转入 LangGraph 运行器，负责数据库准备、异步图执行和运行结果日志记录，是自然语言任务进入 Agent 状态图的主要边界。

代码 4-2 Memory Checkpoint 后端选择 (composition/graph/checkpoint.py)

```

backend = str(config.graph_checkpoint.backend or "redis").strip().lower()
if backend == "memory":
    global _MEMORY_CHECKPOINTER
    if _MEMORY_CHECKPOINTER is None:
        from langgraph.checkpoint.memory import InMemorySaver
        _MEMORY_CHECKPOINTER = InMemorySaver()
    yield _MEMORY_CHECKPOINTER
    return

```

这段代码使本地 Demo 可以通过 GRAPH_CHECKPOINT_BACKEND=memory 运行，不再强制依赖 Redis 服务，解决了课程展示环境中 Redis 配置成本较高的问题。

代码 4-3 Memory URL Channel 工厂 (contexts/collection/infrastructure/channel/factory.py)

```

resolved_mode = normalize_pipeline_mode(config.pipeline.mode if mode is None else mode)
if resolved_mode == "memory":
    return MemoryURLChannel()

manager = RedisQueueManager(
    host=config.redis.host,
    port=config.redis.port,
    password=config.redis.password,
)
return RedisURLChannel(manager=manager, key_prefix=key_prefix)

```

URL Channel 负责在 URL 收集和详情页抽取阶段传递任务。memory 模式用于单机串行运行，Redis 模式保留给后续多 worker 或长任务部署。

代码 4-4 Bangumi Demo 启动脚本 (start_bangumi_demo.cmd)

```

chcp 65001 > nul
set "PLAYWRIGHT_BROWSERS_PATH=%~dp0.pw-browsers"
set "DATABASE_URL=sqlite:///output/autospider.db"
set "GRAPH_CHECKPOINT_ENABLED=true"

```

```
set "GRAPH_CHECKPOINT_BACKEND=memory"
set "PIPELINE_MODE=memory"
set "LOCAL_SERIAL_MODE=true"
.\.venv\Scripts\autospider.exe chat-pipeline --target-url-count 5 --max-pages 1 -r "Collect
5 anime entries from Bangumi calendar page https://bangumi.tv/calendar."
```

启动脚本将编码、浏览器目录、数据库、checkpoint 和 pipeline 模式统一固定，降低了 Windows 环境下手动执行命令时的出错概率。

4.9 AI IDE 使用截图

项目开发和调试过程中使用 VS Code + Codex 插件辅助定位问题。截图展示了在 IDE 中查看 AutoSpider 源码、分析 Redis 连接失败日志，并根据 Agent 建议定位 pipeline mode 配置问题的过程。

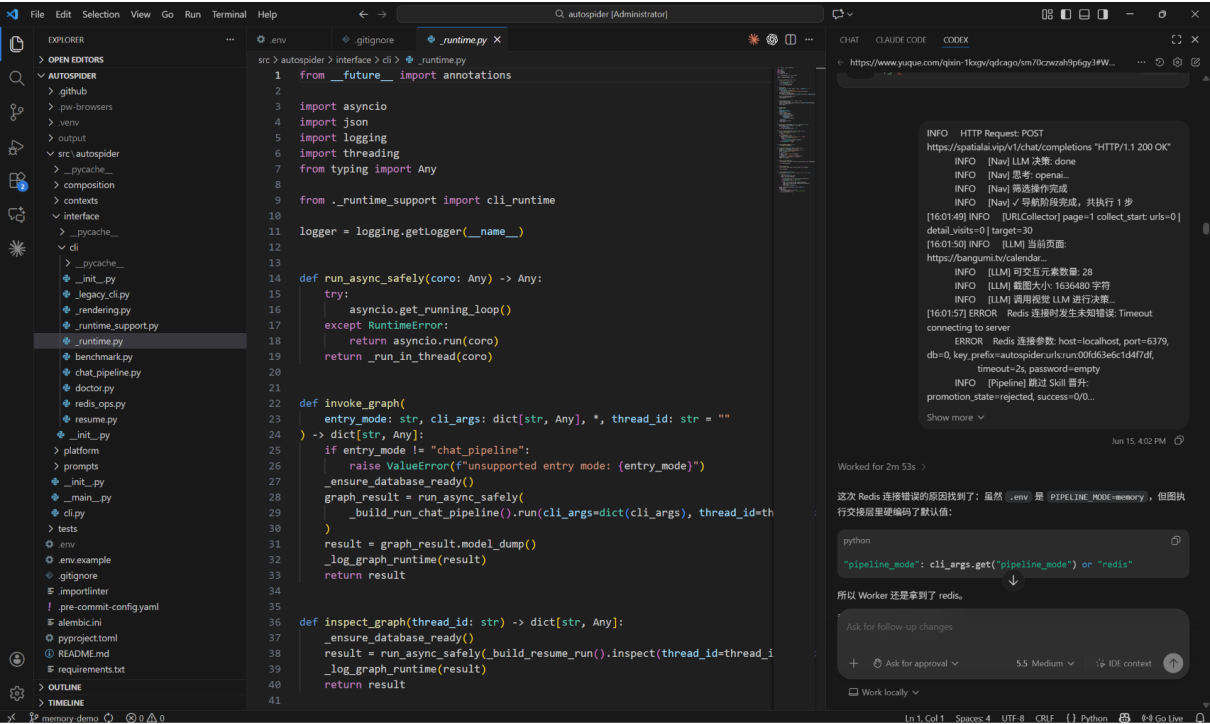


图 4-1 AI IDE 辅助调试与代码修改截图

五、测试与评估

5.1 测试环境

项目	配置
操作系统	Windows
Python	Python 3.11

浏览器	Playwright Chromium
数据库	SQLite
LLM	OpenAI 兼容 API
运行模式	memory checkpoint + memory pipeline
测试网站	https://bangumi.tv/calendar

5.2 功能测试

测试项	测试方法	预期结果	状态
环境启动	执行 start_bangumi_demo.cmd	CLI 正常启动并生成任务配置	通过
任务澄清	输入 Bangumi 采集需求	自动识别 URL、字段、数量	通过
页面导航	Playwright 打开 calendar 页面	浏览器正常打开目标页面	通过
URL 收集	从页面候选元素提取详情页链接	生成 collected_urls.json	通过
字段抽取	访问详情页抽取标题和星期	生成 pipeline_extracted_items.jsonl	部分稳定
结果聚合	合并子任务输出	生成 merged_results.jsonl	通过

5.3 Agent 行为评估

评估维度	指标	当前表现
任务完成率	是否完成端到端链路	可以完成小规模 Demo
运行耗时	5 分钟内产出结果	小规模任务可控
工具调用	是否真实操作浏览器	使用 Playwright 完成页面访问

状态管理	是否支持 checkpoint	支持 memory checkpoint
可观测性	是否输出日志和摘要	输出 runtime.log 与 summary
稳定性	多次运行结果波动	受视觉 LLM 和页面结构影响

5.4 课程核心技术要素覆盖

核心技术要素	覆盖情况
SDD 规格驱动开发	通过 Product Spec、Architecture Spec、CLI Spec 体现
工具使用 / Function Calling	Playwright 浏览器工具、LLM 决策工具调用
记忆机制	LangGraph checkpoint、history match、memory backend
状态管理与多步骤推理	LangGraph 多阶段状态流
多模块协作	chat、planning、collection、experience 多上下文协作
可观测性与评估	runtime.log、task_plan.json、pipeline_summary.json

5.5 当前问题分析

- 视觉 LLM 调用成本较高，页面截图较大时 token 消耗明显。
- 页面文本匹配存在波动，同一任务多次运行可能收集到不同数量的 URL。
- 字段抽取结果有时会包含冗余文本，需要后处理进一步清洗。
- 当前展示主要依赖 CLI 和文件输出，缺少 Web 可视化页面。

5.6 Demo 结果说明

当前 Demo 的目标不是证明系统已经达到生产级爬虫性能，而是验证 Agentic AI 采集链路能够端到端运行。以 Bangumi 每日放送页面为例，系统能够从自然语言任务出发，打开页面，规划采集目标，收集详情页 URL，并将结果写入本地文件。

在小规模目标数量下，系统可以稳定产生 1 至 2 条有效记录。由于页面理解依赖视觉模型，且每次导航和截图都需要模型判断，运行耗时明显高于传统固定规则爬虫。这说明项目更适合展示“通用化采集 Agent”的技术路径，而不是替代所有高性能爬虫场景。

结果文件主要包括：

文件	内容
output/merged_results.jsonl	最终合并后的结构化结果
output/subtask_leaf_001/collected_urls.json	收集到的详情页链接
output/subtask_leaf_001/pipeline_summary.json	成功数、失败数、运行摘要
output/task_plan.json	Planner 生成的任务计划
output/plan_knowledge.md	规划阶段沉淀的知识说明
output/runtime.log	运行日志

六、系统升级与扩展

6.1 可扩展架构

当前系统已经按上下文分层，可以继续扩展更多垂直采集场景。例如电商商品列表、新闻列表、招聘岗位列表、论文页面等，都可以复用任务澄清、页面规划、浏览器导航和字段抽取链路。

6.2 下一阶段计划

- 增加 Web 前端，用于展示任务进度、日志、结果表格和截图。
- 增强 XPath / Skill 复用能力，提升同类页面重复采集稳定性。
- 提供 Redis + PostgreSQL 部署方案，支持多 worker 并发采集。
- 增加 benchmark 测试集，对比不同模型和参数下的成功率。
- 接入 LangSmith / Langfuse 等 tracing 工具，分析 token 成本和失败节点。
- 增加字段清洗模块，减少 LLM 输出中的冗余前缀。

6.3 AI 能力演进路径

阶段 1: 当前 MVP
自然语言任务 -> 视觉页面理解 -> Playwright 工具调用 -> JSONL 输出

阶段 2: 稳定采集
XPath Skill 复用 -> 结果清洗 -> Benchmark 评估

阶段 3: 可视化平台

Web 前端 -> 任务队列 -> 日志追踪 -> 结果表格

阶段 4: 生产级 Agent

多 worker -> Redis/PostgreSQL -> Tracing -> 安全沙箱 -> 自动评测

七、课程总结

7.1 个人收获

通过本项目，我对 Agentic AI 应用开发有了更完整的理解。一个可运行的 Agent 系统并不是简单调用 LLM API，而是需要把任务规划、工具调用、状态管理、失败恢复、日志追踪和结果评估组织成完整工程链路。

本项目让我理解到，LLM 擅长理解目标和生成决策，但真正完成任务还需要可靠的工具系统。Playwright 浏览器工具让 Agent 能够接触真实网页，而 LangGraph 状态流让多步骤执行过程更可控。

7.2 工程思维转变

- 从“写固定爬虫”转向“构建可泛化采集 Agent”。
- 从“模型回答”转向“模型规划 + 工具执行”。
- 从“跑通功能”转向“关注可观测性、可恢复性和可评估性”。
- 从“依赖完整基础设施”转向“根据演示场景降低部署复杂度”。

7.3 对课程的理解

本课程强调企业级应用软件设计与开发，不仅关注功能实现，也关注工程结构、技术路线、部署方式和文档规范。AutoSpider 项目覆盖了 AI Agent 系统从需求到运行结果的完整链路，也体现了 SDD 规格驱动、工具调用、状态管理和测试评估等课程核心要求。

附录：仓库目录结构

```
cs599-project/  
├─ docs/  
│   └─ architecture.md  
│   └─ CS599_大作业报告.docx  
├─ src/  
│   └─ autospider-project/
```

```
|      |─ src/autospider/
|      |─ tests/
|      |─ start_bangumi_demo.cmd
|      |─ start_autospider.cmd
|      |─ pyproject.toml
|      └─ README.md
|─ README.md
|─ LICENSE
└─ .gitignore
```

参考资料

- AutoSpider 项目代码与本地改造记录。
- LangGraph 官方文档。
- LangChain 官方文档。
- Playwright 官方文档。
- OpenAI Compatible API 文档。
- Bangumi 每日放送页面: <https://bangumi.tv/calendar>